

Phase 4
Final Report
Dance Dance Karaoke

Tuesday Group 7

Andy Payan (apaya014)

Z Wang (zwang577)

Edward Almaraz (ealma014)

Hanson Pan (hpan047)

Github Repositories

<https://github.com/edwardalmaraz/Dance-Dance-Karaoke-CS131> (Main Repository)

- The link above is the main repository for our project. The repository contains a README file that explains our Docker container and provides instructions for running the code. We needed to develop several workarounds for dependencies that could only be run within Docker.
- In addition, we ended up sharing a single Jetson Nano for much of the development process. As a result, our commit history may not accurately reflect each individual's contributions, since we frequently had to switch between Git user accounts when committing and pushing changes from the device. This process was sometimes unreliable and led to inconsistencies in the recorded commit history. With that said, all members of the group agree that the workload was distributed evenly and that each team member contributed an equal amount of effort throughout the project.

<https://github.com/ZWang0987/pose-model-cs131.git> (Side Posenet Repository)

- The link above is a secondary repository for PoseNet, the AI-based human pose estimation model used for our pose detection and comparison system. This repository was primarily used by Z during the initial development phase before the PoseNet integration and pose comparison components were merged into the main project repository.

Poster Link

<https://canva.link/e53awvhig7v3fpr>

Demo

Our team did a live demo during the poster presentation on Tuesday. Here is a small recording as proof:

<https://drive.google.com/file/d/1OsxmDJyNnGmomxHK5dxVkcWSiYiVf9hc/view?usp=drivesdk>

Background / Use Case

Karaoke bars in recent times have become an expensive outing for many people, and even an outdated one. Most karaoke bars overcharge their room rentals, along with any food and drink that the customers may want to indulge in, all while the equipment is outdated and offers no updates to their music catalog. On top of this, as time goes on, music and dancing are becoming more and more intertwined. There are ways to enjoy dancing, such as ‘Just Dance’ video games, although this has also become an expensive outing for most people as well. Just Dance offers the base game with limited song selections for around \$30, but to get full access to the whole catalog, you must pay an additional monthly subscription. The mediums of the dancing/singing entertainment industry will only continue to become more popular, as genres such as K-pop, which combine both singing and dancing, that cater to a wide and younger audience, continue to grow. With this growing popularity also comes the demand for more products/outings to cater to these needs, but as we’ve mentioned, these companies will continue to offer subpar experiences to their customers and will gladly continue charging a hefty fee as well.

Dance Dance Karaoke is an open-source dance and karaoke machine that uses the Jetson Nano as the edge device and utilizes GCP as the cloud platform. Our system enables real-time dance and karaoke gameplay, in which users must perform a sequence of poses displayed. In addition, our system plays music and displays lyrics in real time. We then perform pose, lyric, and pitch comparisons during post-processing, returning a total score to the user for each song. Our system is also designed to stay up to date with the most popular songs and dance choreographies. This allows the edge device to select songs from an expanding catalog. Finally, we host a leaderboard that lets users compare their scores with those of other players. Our distributed edge system was tested with two Jetson Nanos; however, the dynamic scalability of the system enables multiple clients to play simultaneously.

Solution Overview

At the edge, Dance Dance Karaoke will output the song lyrics, song audio, song choreography, a visual display of the user's human segmentation model, and the leaderboard scores for each song. The edge device will also record player audio, perform speech-to-text transformation, and conduct player choreography comparison. The user will select which song they want to perform at the edge, which will then fetch the selected song and its related metadata from the server.

On the server, all songs and their associated metadata, such as audio, lyrics, pitch, and choreography, will be stored. The server will also compute lyric accuracy, pitch accuracy, and a total score for each song performed by the user. In addition, the leaderboard will be hosted on the server and can be fetched by the edge device.

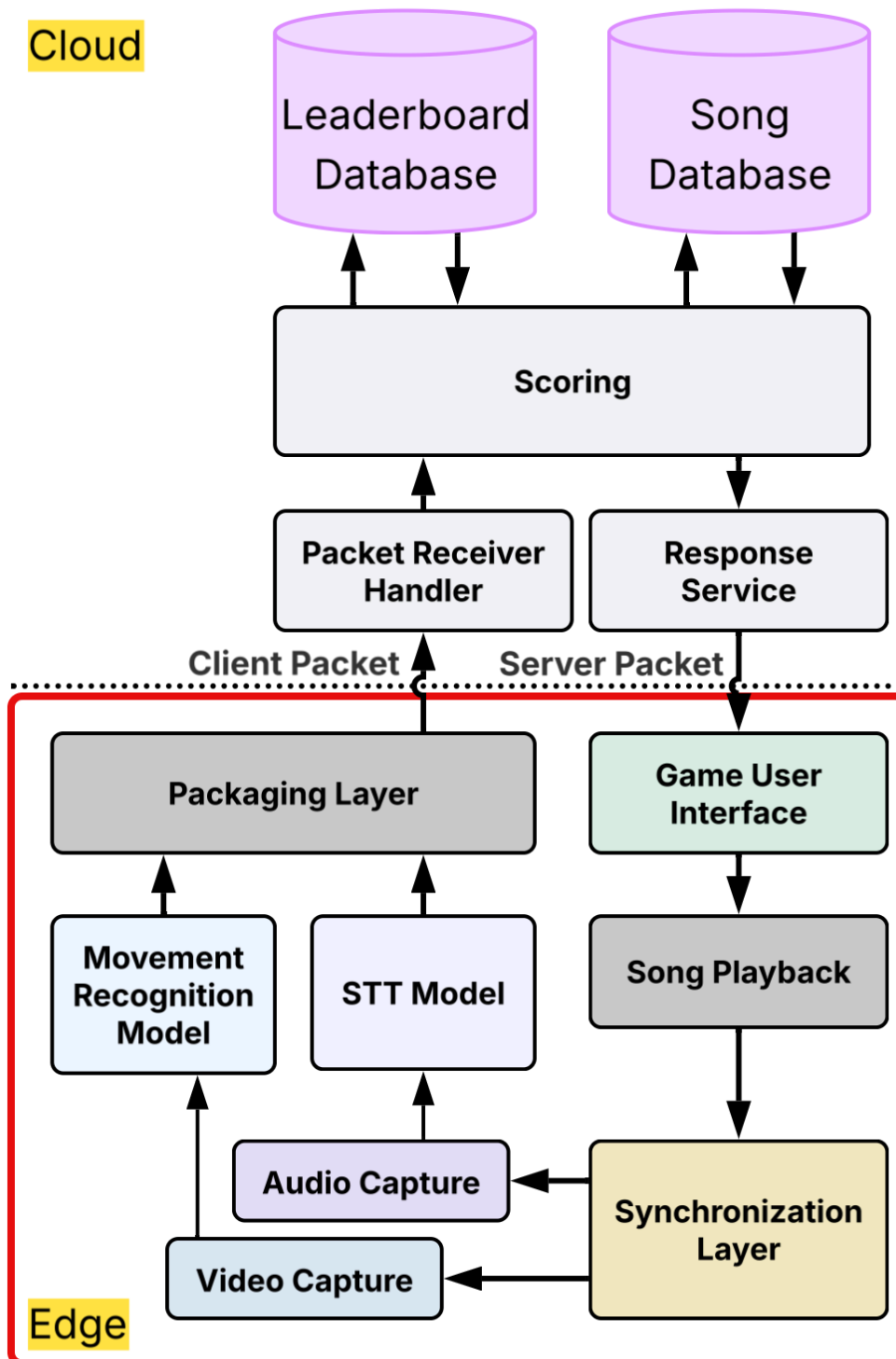
Dance Dance Karaoke consists of two main AI models: PoseNet, an open-source human pose estimation model used for choreography comparison, and Whisper, an OpenAI audio model used for speech-to-text transformation and pitch comparison. PoseNet is run strictly on the edge device, where it captures the locations of every player's joint every five seconds during runtime and saves the data for post-processing choreography comparison.

During runtime, our system records player audio as a .wav file. During post-processing, Whisper is used at the edge for speech-to-text transformation. After obtaining the post-processed choreography score and speech-to-text file at the edge, the choreography score, audio .wav file, and speech-to-text file are sent to the cloud. The cloud then performs pitch comparison by comparing the recorded .wav file against the .wav file of the originally selected song using Whisper. We also use a per-character matching algorithm to compare lyric accuracy.

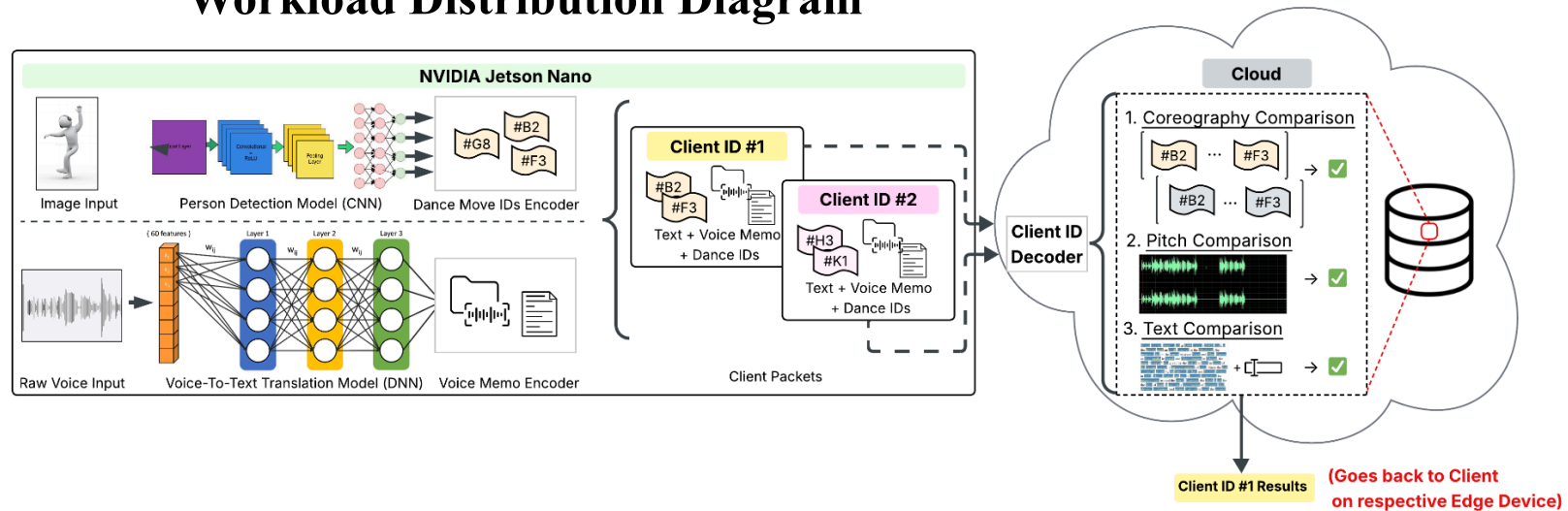
The server then computes a total score for the user based on the choreography score, pitch score, and lyric score. This score is saved to the online leaderboard and is also sent back to the edge device for local display.

Dance Dance Karaoke is also a distributed system that allows multiple users to play simultaneously. This is accomplished through our player ID system, where users enter a player ID before starting a song so their score can be associated with their account for that song. This allows multiple users to perform the same song at the same time, receive real-time score feedback, and have their scores stored on the leaderboard simultaneously.

Device System Design Diagram



Workload Distribution Diagram



EDGE Workload

Input	Computation	Output
- Record player movements via camera sensor - Record player audio via audio sensor - Server packets consisting of: - Final player score - Text accuracy - Pitch accuracy - Choreography accuracy - Score relative to online leaderboard	- Takes recorded player movement and transforms it into a person segmentation frame using a CNN. - Takes recorded player audio and extracts the text and pitch waveforms using a DNN. - Encodes player segmentation frames, speech-to-text data, and vocal pitch information. - Extracts the accuracy of player segmentation frames, speech-to-text processing, and pitch detection.	- Displays dance choreography along with the song's lyrics. - Client ID packets consisting of: - Unique client ID's - Player segmentation frames data - Encoded speech-to-text data - Encoded pitch data

CLOUD Workload

Input	Computation	Output
- Client ID packets consisting of: - Unique client ID's - Player segmentation frames data - Encoded speech-to-text data - Encoded pitch data	- Extracts player segmentation frame, encoded speech-to-text data, and encoded pitch data, and labels them according to the corresponding unique client ID. - Decodes speech-to-text data and pitch data. - Computes the accuracy of player segmentation frames compared to the song choreography segmentation sets. - Computes the accuracy of player speech-to-text and pitch compared to the song lyrics and pitch. - Updates global leaderboard	- Server packets consisting of: - Final player score - Text accuracy - Pitch accuracy - Choreography accuracy - Score relative to online leaderboard

Technical Worksheet (Tool Inventory)

Edge Layer:

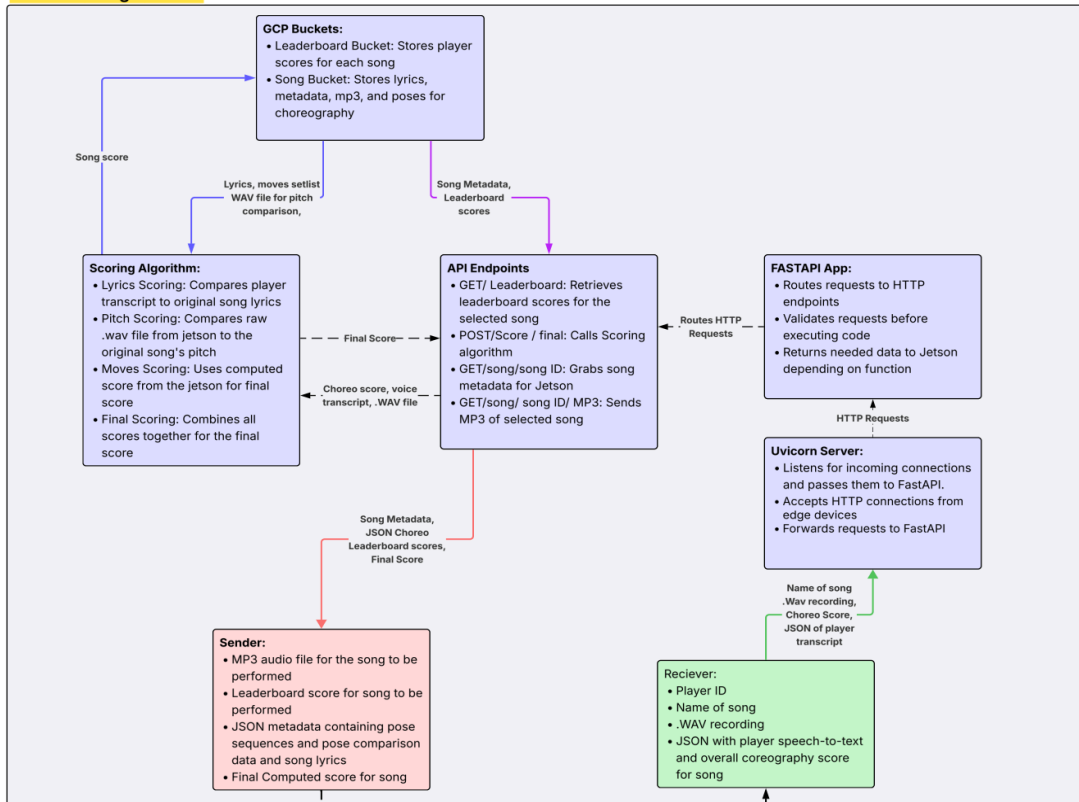
- Jetson Nano: main edge unit used for audio/visual computations from its inputs
- Microphone (audio sensor): Constantly captures user's voice, used as input for DNN model for voice-to-text
- WebCam (camera sensor): Collect video frames of the user's real-time movements, used as inputs for CNN model for model creation
- CNN (Person Detection Model): ML model that transforms video frames into player segmentation models
- DNN (Voice-To-Text Translation Model): ML model that translates audio files into text, along with extracting pitch waveforms
- Client Packets: Computed information from both models, unique client IDs, player segmentation frame data, encoded speech-to-text data, and encoded pitch data, all packaged together, ready to be shipped to the cloud layer.

Cloud Layer:

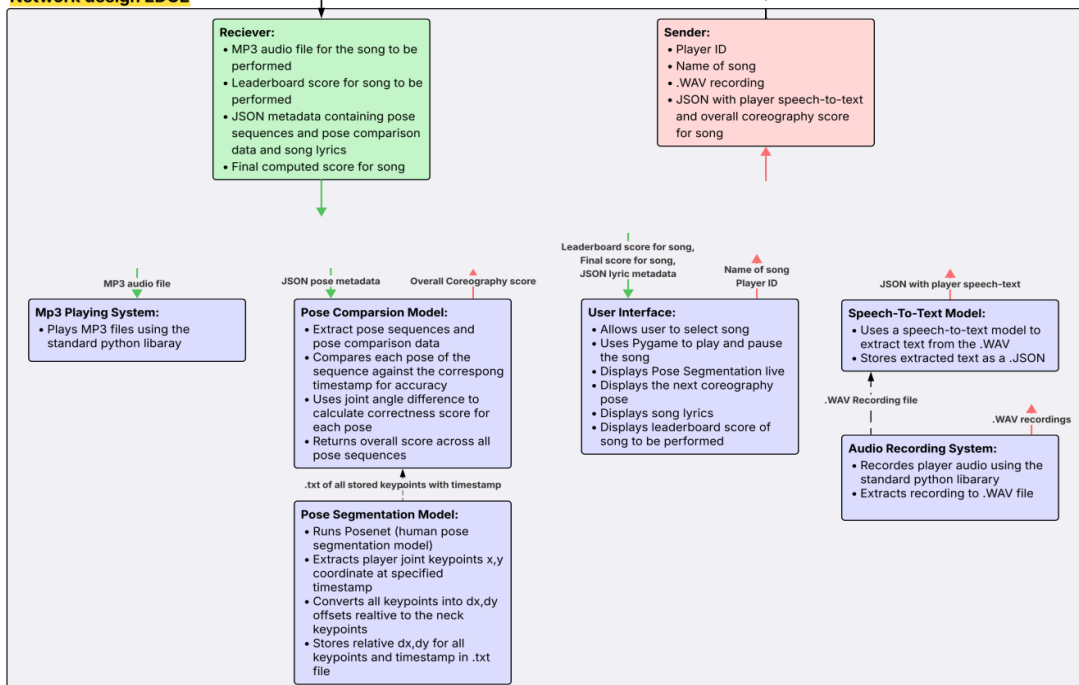
- Cloud Server: Used to unpack client packets for evaluating users' game inputs
- Leaderboard Database: Stores high scores of each song associated with the respective user.
- Song Database Stores song MP3s, choreography, and lyrics.
- Server Packets: Encoded data that consists of the final player score, text accuracy, pitch accuracy, choreography accuracy, and score relative to online leaderboard sent to the edge device for outputting relevant information to the UI.

Network Diagram

Network design CLOUD



Network design EDGE



Device catalog with hardware features

Edge Layer:

- Jetson Nano
 - Uses PoseNet for real-time human pose segmentation and joint/keypoint extraction
 - Computes joint-angle difference scores for pose accuracy evaluation and performs timestamp-based pose comparisons against the song choreography sequence to generate an overall choreography performance score
 - Handles MP3 playback
 - Records player audio input and converts recordings into the .WAV format. Runs speech-to-text processing on recorded .WAV files using Whisper
 - Displays live pose guidance, lyrics, and leaderboard information through the UI

Cloud Layer:

- Cloud
 - Uses Google Cloud Run to host our FastAPI application in a Docker container, which scales automatically based on current traffic. It basically serves as a way to run our HTTP API endpoints created with FastAPI
 - Uses FastAPI as its framework for the API endpoints, which helps route the HTTP requests that are sent from the Jetson Nano to compute or send any needed game data
 - Uses Uvicorn server for forwarding HTTP requests to the FastAPI application
 - Computes the final weighted score using the scoring algorithms that are housed in the FastAPI application.
 - Computes lyric accuracy via a per-character matching algorithm and pitch accuracy using Whisper AI Model
 - Stores leaderboard scores according to the song ID and player ID
 - Allows for the retrieval of stored information that the Jetson needs for gameplay, using the buckets made from Google Cloud storage
 - Uses buckets from Google Cloud Storage to house player leaderboard scores and everything related to the song catalog. This is so the cloud can easily handle data retrieval requests from the Jetson in real time

Network Report

Services Used:

- FastAPI - Web framework for the creation of our server. Houses HTTP requests that go back and forth between the Jetson Nano and the cloud.
- Google Cloud Run - Cloud service that hosts our FastAPI application in a Docker container.
- Google Cloud Storage - Google storage buckets used to house needed data for the songs (lyrics, choreography, metadata), as well as leaderboard information to help track which players have the highest scores on specific songs. The buckets are formatted to store MP3s, JSONs, and text files for each song.

Quality Attributes:

Our network is guaranteed reliability as one of its key attributes, with our fastAPI server's incorporation in Google Cloud Run, which automatically restarts crashed containers and scales dynamically under different load sizes. We have type validation on all POST endpoints, which can ensure data integrity by rejecting malformed requests before anything is fully executed. All communication between the Jetson Nano and the cloud is encrypted over HTTPS, and Cloud Run handles certifications to access these endpoints. Player data is stored securely on the GCP buckets and can only be accessed from an authenticated user or an administrator, so security issues relating to that are not a problem either. Cloud Run also automatically runs the communication protocol with TLS (Transport Layer Security), which encrypts everything, adding a layer of security.

Messaging patterns and communication protocols:

We have implemented a few messaging patterns for our network, with the majority of them being between the Jetson and the FastAPI server that resides on our Google Cloud services.

From the Edge(Jetson) to the Cloud(Server):

- 1.) Jetson sends the Player's ID and the song ID when the user selects a song to perform.
- 2.) Jetson records and compares the player pose sequence to the song sequence within the JSON file received from the cloud and computes a score for the player's choreography. This choreography score in JSON is then sent to the cloud.

- 3.) Jetson plays the MP3 file sent from the server, records the player audio, and converts it to a .WAV file and a JSON file with the pure speech-to-text of the recorded player audio. Both the .WAV and JSON are sent to the server.
- 4.) Jetson receives the total score from the server and displays it to the user.
- 5.) The user wants to see the leaderboard scores, so the Jetson sends a leaderboard score request to the server.

From the Cloud(Server) to the Edge(Jetson):

- 1.) Server sends out a packet containing an MP3 file with respect to the song name received and a JSON file containing the choreograph sequence and poses needed in the sequence and the song lyrics.
- 2.) Server receives the overall player choreography score from the Jetson
- 3.) Server receives the .WAV file and JSON file of player audio from the Jetson and computes a lyric and pitch score for the respective song.
- 4.) Server computes a total score for the song that includes the player's choreography, lyrics, and pitch. This score is then saved in the cloud leaderboard for the respective song and sent back to the Jetson for display.
- 5.) Server grabs leaderboard data and sends it to the Jetson in a JSON format

The communication protocol is HTTPS; this is how the cloud communicates with the Jetson Nano. This is done via a classic request-response model, where the Jetson sends requests and any necessary information to complete them, and the cloud processes all requests and responses over HTTPS. This is done when the Jetson sends an HTTP request to the cloud URL; the cloud processes the request and returns the necessary game data.

Acknowledgements

For this project, there were components where AI was used, as we are not Python experts. For example, we were not familiar with Pygame, which was used to build our user interface, so we utilized AI assistance for that component. With that being said, the overall idea for the project, as well as how the system was designed to operate and how the various computations were performed, were our own original ideas. For example, our pose calculation method based on angle differences was mathematically theorized and designed beforehand. AI was primarily used to assist with translating those mathematical concepts and algorithms into Python code and resolving specific syntax issues.

